Scaling Laws for Transformer Language Models on SVG Code

Introduction

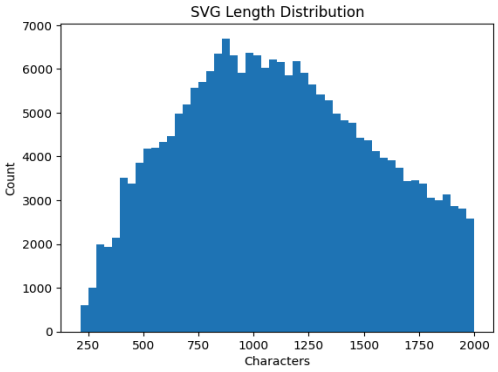
Scaling laws - the empirical observation that validation loss follows a predictable power law as a function of model size and training compute - have become a cornerstone of modern deep learning [1], yet have been studied almost exclusively on natural language. It remains an open question whether similar regularities emerge in structured, non-linguistic domains with fundamentally different statistical properties.

In this work, we investigate scaling laws for decoder-only transformers trained on Scalable Vector Graphics (SVG) code. SVG encodes visual content as structured XML text with strict syntactic rules and precise coordinate semantics, making it an unusually clean testbed: a generated sample either passes as valid XML and renders correctly, or not. We train models ranging from ~1M to ~88M parameters, fitting power-law scaling curves to validation loss as a function of model size. A central challenge in such studies is hyperparameter transfer - the learning rate optimal for a small model is generally suboptimal for a larger one under Standard Parameterization (SP). We therefore compare a fixed learning rate tuned on the smallest model against Maximal Update Parameterization (μP) [3], which enables zero-shot learning rate transfer across model widths. Github repo at the end of references.

Data

Preprocessing Pipeline

Our data draws from three HuggingFace datasets released by the StarVector project [4]: all of svg-icons-simple; all of svg-emoji-simple; and 20% of svg-stack-simple. Each SVG was passed through a multi-stage cleaning pipeline: XML comments were stripped, whitespace collapsed, floating-point coordinates rounded to one decimal place, and double quotes normalized to single quotes. SVGs were then filtered to the character length range [50, 2000], with the upper bound chosen to keep sequence lengths manageable for smaller models. Each remaining SVG was validated as parseable XML using lxml.etree and render-validated using CairoSVG; samples failing either check were removed. After filtering, the cleaned corpus contained 220,488 SVGs, down from 330,475 raw samples. The pipeline removed approximately 33% of the data. Average SVG length seems to be ~1,112.



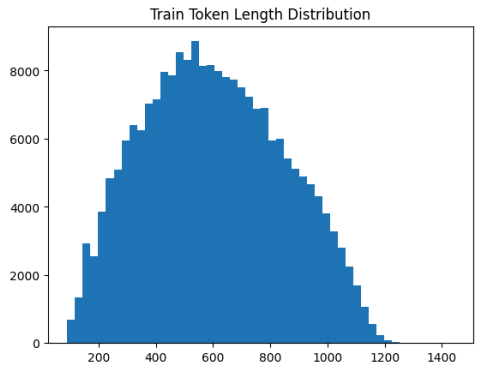
Train/Validation/Test Split

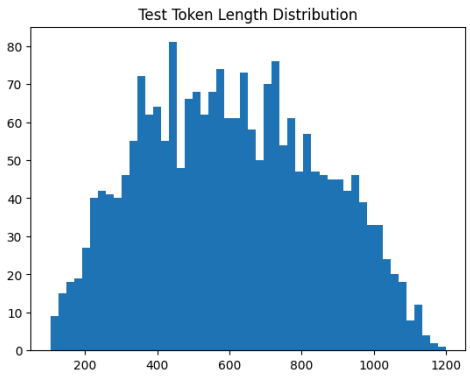
Split	Samples	Tokens
Train 98%	216,078	130,335,440
Validation 1%	2,205	1,349,726
Test 1%	2,205	1,336,593
Total	220,488	133,021,759

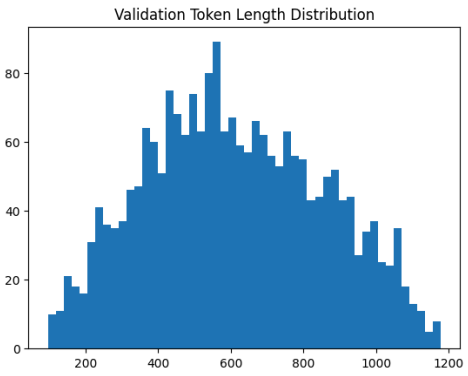
We split by file rather than by token position to ensure no single SVG appears across splits, preventing data leakage.

Tokenization

We trained a Bye-Level BFE tokenizer using the HuggingFace tokenizers library on the training split. Byte-level BPE ensures the tokenizer handles any character sequence without unknown tokens, important for SVG's mix of XML syntax, numeric coordinates, hex color codes, and attribute names. We chose a vocabulary size of 4,000 tokens, which captures the recurring structural patterns (path commands, attribute names, coordinate subwords) without overparameterizing the embedding table relative to our smaller models. Larger vocabularies were considered but deemed unnecessary given that SVG's syntactic diversity is far more limited than the natural language. Four special tokens were included: <pad>, <unk>, <bos>, and <eos>. After tokenization, sequences exceeding 2,048 tokens were discarded, leaving an average sequence length of ~603 tokens - a compression ratio of ~1.8 characters per token, reflecting BPE's ability to merge common SVG substrings into single tokens.







SVG Dataset Examples

Methods

Architecture

We implemented a decoder-only transformer following the nanoGPT architecture [5], consisting of casual self-attention blocks with per-layer normalization, GELU activations, and a linear language modeling head. We trained five different model sizes by varying embedding dimension, number of layers, and number of attention heads, with the feedforward multiplier fixed at 4x the embedding dimension. All models used a context window of 256 tokens and a vocabulary size of 4,000.

For Part 2 (SP), both depth and width varied across model sizes. For Part 3 (μP), depth was fixed at 6 layers across all sizes - a theoretical requirement of μP's set_base_shapes which infers per-layer learning rate multipliers by comparing a base and delta model that differ exclusively in width. Varying depth would make this comparison ambiguous and break the transfer guarantees. We also replaced the standard $1/\sqrt{d}$ attention scaling with $1/d$, replaced the output projection with MuReadout, and used MuAdamW as the optimizer. Part 4 uses the μP XL configuration.

Model	Parameters SP vs μP	d_model	n_layers SP vs μP	n_heads	d_ff
Tiny	1,854,112 vs 2,250,656	128	4 vs 6	4	512
Small	4,258,720 vs 4,258,720	192	6 vs 6	6	768
Medium	13,821,856 vs 13,821,856	384	6 vs 6	6	1536
Large	35,755,936 vs 23,146,400	512	10 vs 6	8	2048
XL	91,400,608 vs 48,873,376	768	12 vs 6	12	3072

Training Setup

For Parts 2 and 3, we first performed a learning rate sweep on the respective Tiny model, testing seven values on a log scale: {1e-5, 3e-5, 1e-4, 3e-4, 1e-3, 3e-3, 5e-3} for 2,000 steps each. Using the best learning rate from the sweep, all models were then trained for one epoch (~130M tokens, 15,910 steps) using a batch size of 32 sequences of 256 tokens, AdamW (Parts 2) or MuAdamW (Parts 3–4) with weight decay 0.1, and a cosine learning rate schedule with 5% linear warmup, on an NVIDIA A100 GPU in Google Colab.

For Part 4, we trained the μP XL model for 3 epochs (~390M tokens, 47,730 steps) as it was the best in Part 3. We made two targeted adjustments relative to Part 3: the peak learning rate was reduced from 5×10^{-3} to 3×10^{-3} to reduce optimizer overshooting, and the warmup period was extended from 5% to 7% of total steps for more gradual ramp-up. A single cosine schedule was applied over the full training horizon rather than resetting per epoch, and gradient clipping at

norm 1.0 was applied throughout. The model was checkpointed based on validation loss, so the final model corresponds to the best observed checkpoint rather than the last training step.

Scaling Law Fit and Extrapolation

After training, we fit a power law of the form $L = a \cdot N^{-\alpha} + c$ to validation loss as a function of parameter count, where α is the scaling exponent and c is an irreducible loss floor, using `scipy.optimize.curve_fit` with initial parameters $a=1.0$, $\alpha=0.05$, $c=0.4$. This was applied to both the Part 2 and Part 3 results. For Part 3, we additionally extrapolated predicted validation loss to a model 10× larger than XL, with uncertainty estimated as the standard deviation of residuals between the fitted curve and observed losses.

Evaluation

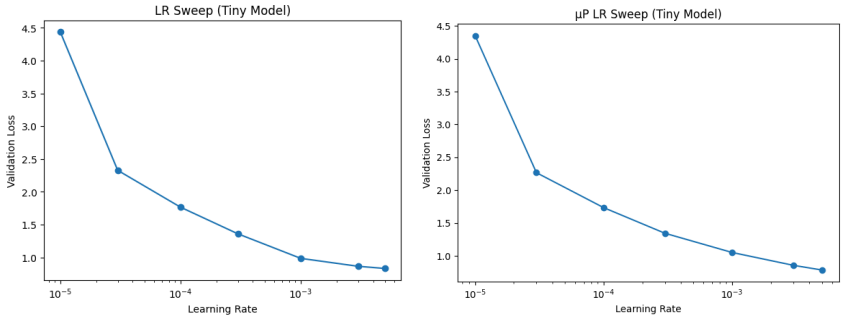
We evaluated the best Part 4 checkpoint on the held-out test set to compute test loss and perplexity. For generation, we produced samples at temperatures {0.5, 0.8, 1.0} combined with top-k (k=50) and nucleus (top-p=0.95) sampling. Each generated sequence was post-processed to reconstruct valid XML from the tokenizer's byte-level output, including collapsing split decimal numbers, rejoining hyphenated attribute names, and normalizing quote spacing. Generated samples were evaluated for XML validity, SVG structural validity, and render success using `lxml.etree` and `CairoSVG`.

Results

Part 2 SP Scaling Study Vs Part 3 μP Scaling Study

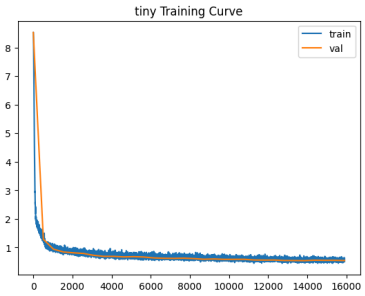
Learning Rate Sweep

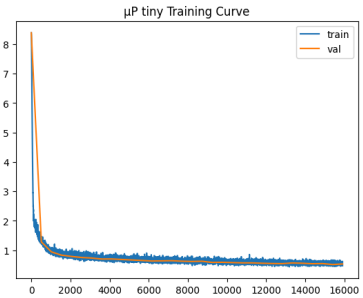
Both parameterizations selected the same optimal learning rate of 5×10^{-3} on the Tiny model. Loss decreased monotonically across the sweep range, suggesting the optimal learning rate may lie at or slightly above 5×10^{-3}, though higher values risk divergence. The μP Tiny model achieved a slightly lower final validation loss than its SP counterpart, suggesting marginally better optimization under μP even at small scale.

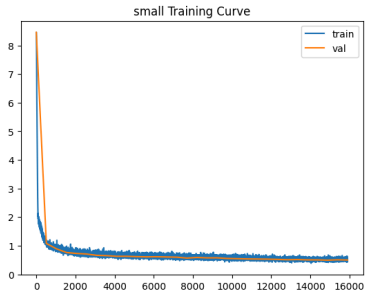


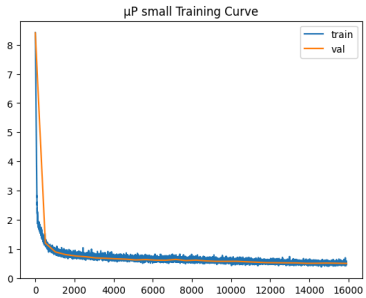
Training Curves

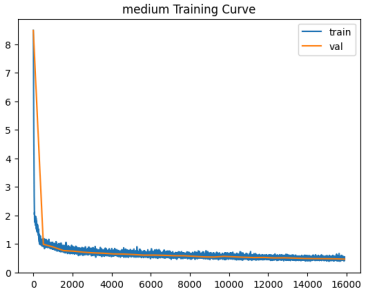
Under SP, Tiny, Small, and Medium followed stable, monotonically decreasing validation loss trajectories. Large and XL exhibited a characteristic three-phase pattern: rapid initial descent during warmup, a prolonged instability phase where loss spikes and stays elevated, followed by recovery once the cosine schedule reduces the learning rate below a stable threshold. Under μP, all five models exhibited stable, monotonically decreasing validation loss throughout, with no instability humps. Occasional sharp spikes are visible in both sets of curves and are attributable to stochastic batch sampling; these are transient and do not affect overall convergence.

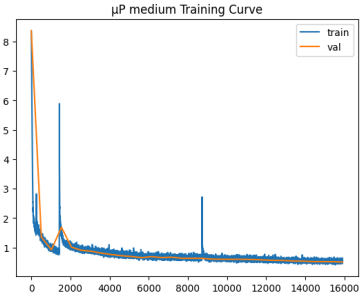


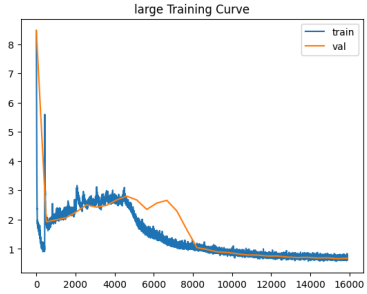


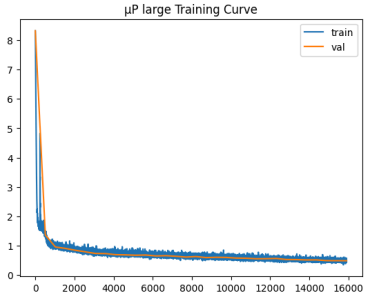


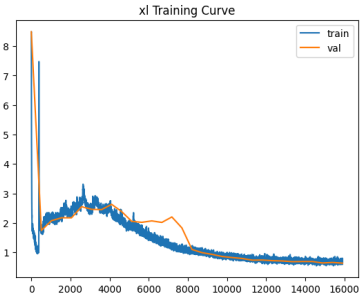


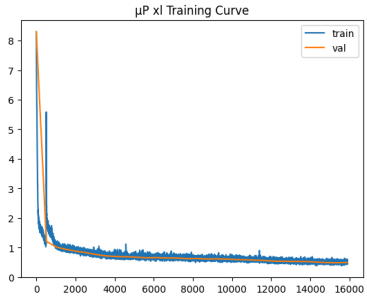








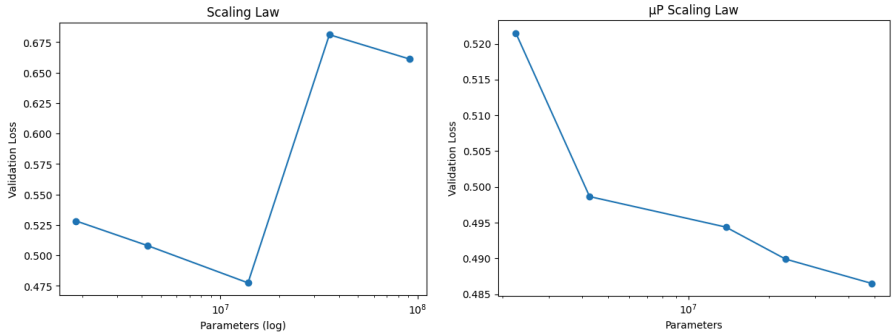




Model	Final Val Loss SP vs μP	Time (s) SP vs μP	Tok/s SP vs μP	GPU Mem (GB) SP vs μP
Tiny	0.5282 vs 0.5215	269.8 vs 365.6	483028.2 vs 356525	1.32 vs 2.54
Small	0.5081 vs 0.4987	485.3 vs 508.8	268571.3 vs 256144	2.09 vs 3.00
Medium	0.4774 vs 0.4944	941.8 vs 967.2	138382.1 vs 134749	2.86 vs 2.82
Large	0.6812 vs 0.4899	2162.5 vs 1402.2	60269.0 vs 92948	6.32 vs 3.58
XL	0.6611 vs 0.4865	5037.0 vs 2685.8	25875.4 vs 48528	12.54 vs 6.19

Scaling Results

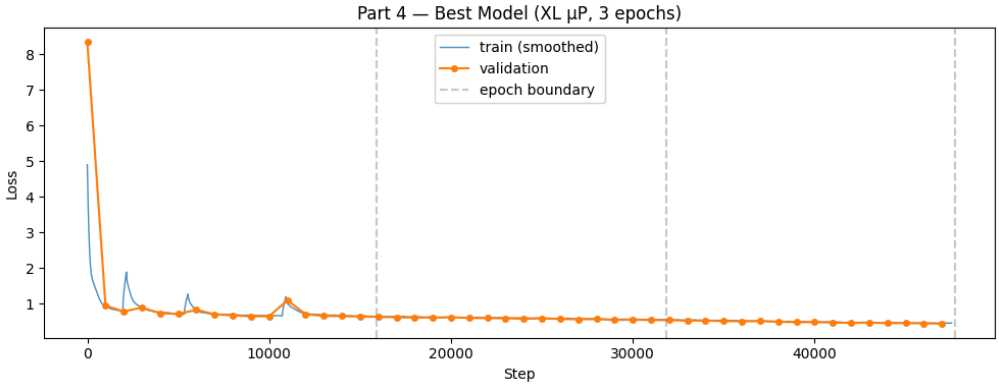
At small models, SP and μP perform similarly; the validation loss gap between SP and μP is only 0.0067 at Tiny and 0.0094 at Small. The difference becomes dramatic at larger scales; SP Large and XL are actually worse than SP Medium, while μP maintains consistent improvement across all sizes. The SP power law fit failed to converge meaningfully under the non-monotonic curve, yielding a degenerate exponent of α=900260. The μP curve supports a meaningful fit: $a=3.23\times10^{9}$, α=1.73, c=0.4893. The large exponent is attributed to the one-epoch compute constraint - larger models are increasingly undertrained relative to their capacity, causing early saturation. Extrapolating to a ~489M parameter model predicts a validation loss of 0.4893±0.0022, essentially at the estimated loss floor, confirming that further parameter scaling without additional training tokens yields negligible improvement under this compute budget.



Part 4: Best Model Training

Training

Validation loss decreased steadily across all three epochs, from ~0.64 at the end of epoch 1 to ~0.45 by the final steps of epoch 3, with the best checkpoint saved at step 47,000 (val loss 0.4434).



Model	Params	Final Val Loss	Time (s)	Tok/s	GPU Mem (GB)
XL	48,873,376	~0.4434	8118	48163	5.55

Test Set Performance

Evaluating the best checkpoint on the held-out test set yields a test loss of 0.4510, and a test perplexity of 1.57. The perplexity value reflects the model operating over a 4,000 token BPE vocabulary on structured SVG test; direct comparison to natural language perplexities is not meaningful, but the low value indicates the model has learned highly predictable token sequences consistent with SVG syntax.

Generation Quality

All 30 unconditional samples achieved perfect XML validity, SVG structural validity, and render success across all three temperatures - a strong result indicating the model has thoroughly internalized SVG syntax. Prefix-conditioned generation was less reliable, with 3 of 5 completions failing XML validation and rendering, attributable to tokenization artifacts and context truncation rather than fundamental model failures (all 5 had correct SVG root structure). Generated samples are in the Discussion section, and their SVG code in MLprojPart4.ipynb.

Condition	n	XML Valid	SVG Root	Render Rate
Unconditional Temperature 0.5	10	100%	100%	100%
Unconditional Temperature 0.8	10	100%	100%	100%
Unconditional Temperature 1.0	10	100%	100%	100%
Prefix-conditioned	5	40% (2/5)	100%	40% (2/5)

Discussion

Scaling Behavior on SVG vs. Natural Language

Our scaling experiments reveal important differences between SVG and natural language scaling behavior. Under standard parameterization, the power law fit failed entirely, yielding a degenerate exponent of α=900,260 due to the non-monotonic loss curve caused by learning rate instability at larger model sizes. Under μP, where training was stable across all model sizes, we obtained a meaningful fit of α=1.73 and a loss floor of c=0.489. This exponent is substantially larger than the 0.05-0.10 range [1] for natural language models. We attribute this discrepancy primarily to the one-epoch computer constraint. Kaplan et al. [1] derive their scaling laws under conditions where models were trained to convergence; our models were trained for a single epoch, meaning larger models are increasingly undertrained relative to their capacity. This is consistent with Chinchilla's central finding [2] that model size and training tokens must scale together; our setup scales model size while holding training tokens fixed, which naturally produces a steeper apparent exponent and earlier saturation.

The fitted loss floor c=0.489 is also informative. Unlike natural language, SVG has a rigid syntactic structure - valid coordinate values, required attributes, strict nesting rules - that

constrains the distribution of tokens heavily. This means a well-trained model can achieve very low perplexity by learning syntactic regularities alone, and the irreducible loss floor reflects the remaining uncertainty over things like specific coordinate values and color choices that are not fully predictable from context. The extrapolation to a 489M parameter model predicts a validation loss of 0.4893±0.0022, essentially at the floor, confirming that under the current compute budget, adding more parameters alone yields negligible further improvement.

Fixed LR SP vs. μP - When Does It Matter?

The comparison between SP and μP is the clearest result of this project. At small model sizes, the two approaches perform nearly identically; the final validation gap between SP and μP Tiny is only 0.5282 vs 0.5215, and at Small it is 0.5081 vs 0.4987. These differences are modest and within the noise of stochastic training. However, the gap grows dramatically with model size: at Large the difference is 0.6812 vs 0.4899, and at XL it is 0.6611 vs 0.4865.

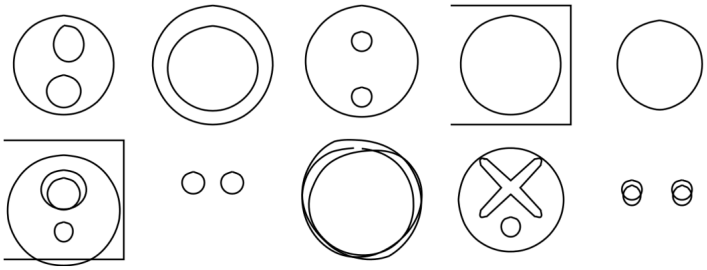
The mechanism behind this is visible directly in the training curves. Under SP, Large and XL exhibit a three-phase trajectory: stable warmup, a prolonged instability phase where loss spikes to ~2.5 and stays elevated for thousands of steps, and eventual recovery as cosine decay brings the learning rate into a stable range. The instability arises because the peak learning rate of $5\text{x}10^{-3}$, which was optimal for the 1.8M parameter Tiny model, is too aggressive for a 35M or 91M parameter model under SP as wider models have larger effective update magnitudes and require smaller learning rates to remain stable. However, under μP, the per-layer learning rate multipliers automatically scale down updates for wider layers, so the same peak learning rate of $5\text{x}10^{-3}$ remains stable across all model sizes. The μP training curves show smooth, monotonically decreasing validation loss for every model including Large and XL, with no instability phase at all.

This result has a practical implication: the difference between SP and μP becomes significant precisely at the model sizes where tuning is most expensive. For a Tiny model, retuning the learning rate is cheap. For a Large or XL model, a full learning rate sweep would require days of compute. μP eliminates this cost by making the small-model sweep valid for all larger models, which is exactly the zero-shot transfer guarantee of Yang et al [3].

SVG-Specific Patterns at Different Scales

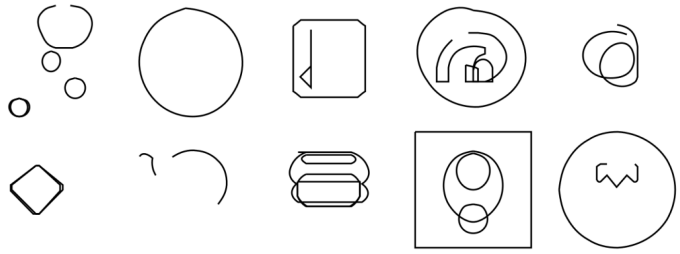
All 30 unconditional samples achieved 100% XML validity, SVG structural validity, and render success across all three temperatures, confirming that syntactic competence is thoroughly established. The model has also internalized the dominant visual convention of the training corpus: nearly every sample uses stroke-only rendering with fill=’none’ and stroke=’black’, reflecting the outline-based icon style that dominates the dataset. Temperature has a striking effect on both diversity and coherence.

Temperature at 0.5



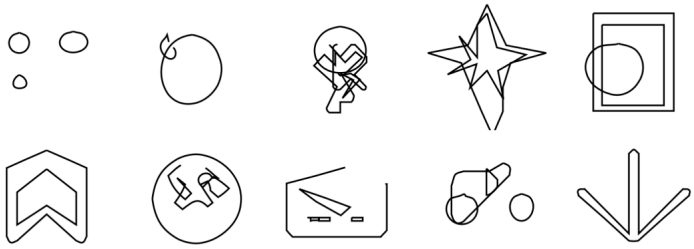
At 0.5, samples are heavily biased toward circles. Concentric circles, circles inside incomplete rectangles, circles with internal circles, and paired small circles appear repeatedly. The model is essentially collapsing toward the most common icon archetypes in the training data. Compositions are spatially coherent but repetitive. One notable sample shows a circle containing an X shape with a small dot below it, demonstrating that the model can produce multi-element compositions with internal structure. However, the variety is limited.

Temperature at 0.8



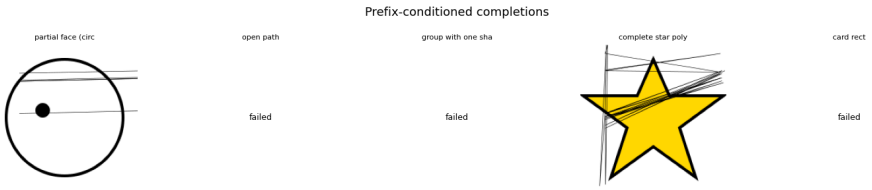
At 0.8, diversity increases substantially. The model now produces a wider range of shape types: a rounded rectangle with a left pointing arrow (maybe a back button), a diamond, a “robot lookalike”, a circle with weird elements inside, and bubbles.These are more icon-like and less stereotype than the temperature 0.5 outputs.

Temperature at 1.0



At 1.0, diversity is highest and the outputs are most visually interesting but also least reliable. The model produces a convincing star shape, a downward arrow, a bookmark-like chevron shape, and more bubbles. However, several samples show over-generation - paths layered on top of each other producing tangled results, such as the dense overlapping strokes in the third sample of the top row. This is consistent with the model sampling less predictable continuations and occasionally failing to terminate a composition cleanly.

Prefix-conditioned Results



This tells a more nuanced story. The partial face prefix produced horizontal lines across the circle rather than a second eye and mouth, which means the model showed no understanding of facial symmetry or the geometric relationship between facial features. This is a clear limitation in spatial reasoning at this scale. The complete star polygon prefix has stray lines drawn on top of it, suggesting the model did not understand what filled in elements are. The remaining three prefixes all failed XML validation and rendering due to tokenization artifacts and context truncation, but not fundamental model failures as all five completions correctly produced SVG root structure.

Limitations and Future Work

The most significant limitation of this study is the compute budget. Training all models for exactly one epoch means larger models are substantially undertrained relative to their capacity, as compute-optimal training requires scaling tokens alongside model size [2]. Directions for future work include: training a tokenizer on pre-normalized SVG to reduce post-processing complexity, exploring longer context windows to better capture complete SVG structure, and conducting a full compute-optimal scaling study where both model size and training tokens are varied jointly.

Conclusion

This project investigated scaling laws for decoder-only transformer language models trained on SVG code, comparing SP against μP across five model sizes.

The central finding is that learning rate transfer is the dominant factor determining whether scaling laws are recoverable in this setting. Under SP, the fixed learning rate tuned on the Tiny model caused catastrophic instability in Large and XL models, producing a non-monotonic scaling curve and a meaningless power law fit. Under μP, training was stable across all model sizes, monotonically decreasing validation loss was seen, and a meaningful power law fit was obtained. The practical implication is clear: for any scaling study where retuning hyperparameters at every model size in infeasible, μP is not just helpful but necessary for obtaining reliable results.

The μP XL model trained for 3 epochs achieved a test perplexity of 1.57 and 100% XML validity and render rate on unconditional samples, demonstrating strong syntactic fluency and consistent adherence to the stroke-based icon conventions of the training corpus.
Prefix-conditional generation revealed the current limits of the model, with spatial reasoning and semantic coherence remaining weak at this scale and compute budget.

References

[1] Kaplan, J., McCandlish, S., Henighan, T., Brown, T. B., Chess, B., Child, R., Gray, S., Radford, A., Wu, J., & Amodei, D. (2020). Scaling laws for neural language models. arXiv preprint arXiv:2001.08361.

[2] Hoffmann, J., Borgeaud, S., Mensch, A., Buchatskaya, E., Cai, T., Rutherford, E., Casas, D., Hendricks, L. A., Welbl, J., Clark, A., et al. (2022). Training compute-optimal large language models. arXiv preprint arXiv:2203.15556.

[3] Yang, G., Hu, E. J., Babuschkin, I., Sidor, S., Liu, X., Farhi, D., Ryder, N., Pachocki, J., Chen, W., & Gao, J. (2022). Tensor programs V: Tuning large neural networks via zero-shot hyperparameter transfer. arXiv preprint arXiv:2203.09789.

[4] Rodriguez, J., Liao, W., Miret, S., & Garg, S. (2023). StarVector: Generating scalable vector graphics code from images and text. arXiv preprint arXiv:2312.11556.

[5] Karpathy, A. (2022). nanoGPT. GitHub repository. https://github.com/karpathy/nanoGPT.

My Github Repo: https://github.com/Nyang-Lin-Phyo/SP26_CS-GY_6923_FinalProject